M-RV32I Processor Core

Multi-Cycle Architecture Reference Manual

Document Version 1.0

Architecture	RISC-V RV32I (Base Integer)
Pipeline	Multi-Cycle (3 to 5 Stages)
Optimization	Area-Optimized, Shared Execution Units
Target	Xilinx 7-Series / UltraScale+ FPGAs
Status	Verified (Simulation)

Date: June 16, 2026

Revision History

Version	Date	Description
1.0	June 2026	Initial release. Document restructured to professional product standards. Included detailed FSM transitions, cycle budgets, and performance metrics.

Contents

1 Product Overview

The M-RV32I is an open-source, resource-efficient CPU core implementing the 32-bit RISC-V Base Integer Instruction Set Architecture (RV32I). Designed specifically for low-area embedded systems and FPGA integration, the core sacrifices single-cycle execution speeds to achieve a remarkably small logic footprint.

1.1 Key Features

- RV32I Compliance: Full support for the base integer instruction set (excluding environment/system calls in this iteration).
- Area-Optimized Architecture: Achieved through aggressive hardware multiplexing. A single Arithmetic Logic Unit (ALU) is shared for PC calculations, branch targets, and standard arithmetic.
- Unified Memory Interface: Operates on a Von Neumann architecture, utilizing a single memory interface for both instruction fetching and data access.
- Variable-Cycle Execution: Instructions complete in the minimum number of cycles required (ranging from 3 to 5), yielding a competitive Cycles Per Instruction (CPI) ratio compared to strict 5-stage unbypassed pipelines.
- 10-State Control FSM: A robust, fully synchronous Finite State Machine manages the entire datapath.

2 Microarchitecture Description

2.1 Datapath Schematic

The core datapath relies on intermediate state registers (`IR`, `A`, `B`, `ALUOut`, `Data`) to latch data between cycles. This allows the shared components (Memory and ALU) to handle different operations safely across the instruction lifecycle.

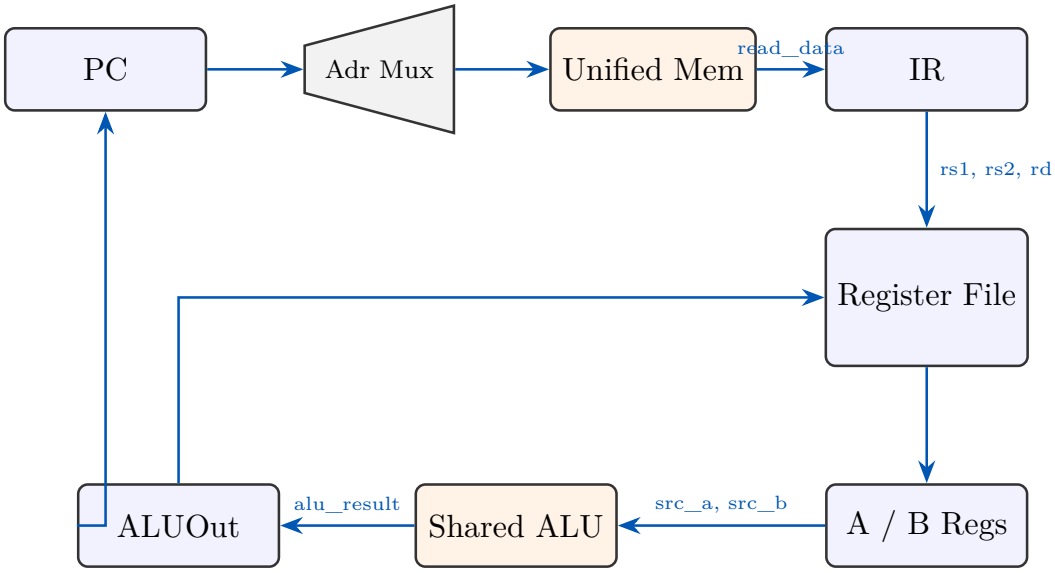

Figure 1: M-RV32I Top-Level Datapath Block Diagram

2.2 Architectural Decisions & Routing Logic

2.2.1 Combinational vs. Sequential ALU Outputs

In the `result_src` multiplexer, both `alu_out` and `alu_result` are available as inputs.

- **`alu_result`** (Combinational): Routed directly back to the PC during the FETCH stage. Because it is combinational, $PC + 4$ is available immediately and latched into the PC on the very next clock edge, avoiding a wasted state.
- **`alu_out`** (Sequential): A state register that safely latches `alu_result` at the clock edge. This is crucial for multi-cycle stability. For example, during MEM_ADR, the ALU calculates a load address. In the next cycle (MEM_RD), the ALU might be idle or repurposed, but `alu_out` securely drives the memory address bus.

2.2.2 JAL Register Write and PC Update Decoupling

The `jal` instruction necessitates updating two state elements simultaneously: the Register File (return address) and the PC (jump target). To prevent bus contention, the paths are decoupled:

- The Jump Target is pre-calculated in DECODE and stored in `alu_out`.
- During the JUMP cycle, the `result` bus carries the return address (the old PC + 4) into the Register File.
- Simultaneously, the PC module ignores the `result` bus and loads the jump target directly from the `alu_out` register via a dedicated internal multiplexer.

3 Execution Control & FSM

The 10-state Finite State Machine acts as the core's nervous system, directing the flow of data through the shared components.

3.1 FSM State Transition Diagram

Figure 2 illustrates the complete execution flow. By separating the diagram into dedicated vertical execution branches, the orthogonal nature of the instruction lifecycles becomes clear.

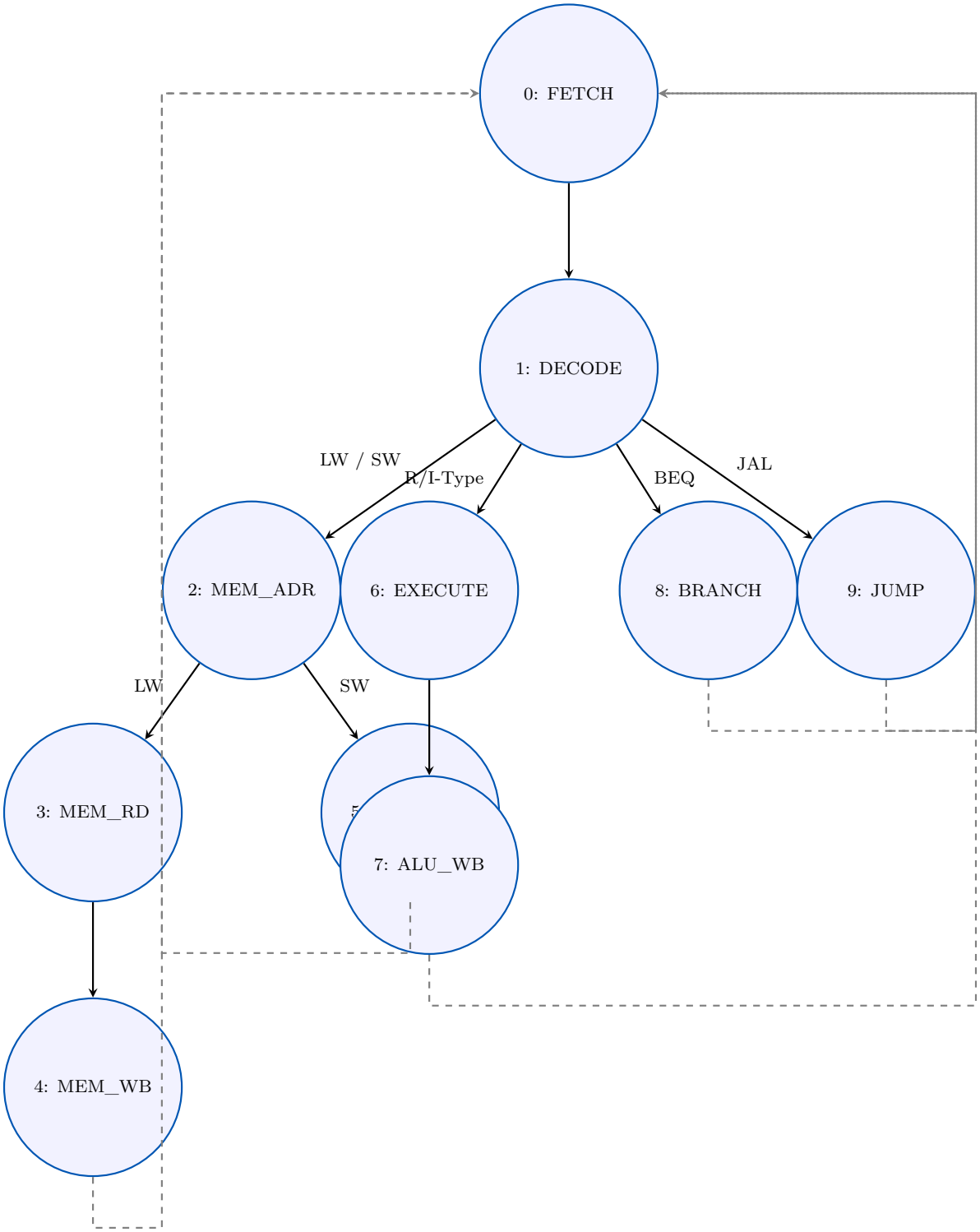

Figure 2: Orthogonal 10-State FSM Control Flow Diagram

3.2 Instruction Cycle Budgets

Instruction Type	State Sequence	Cycles
Load (**lw**)	Fetch → Decode → Mem_Adr → Mem_Rd → Mem_WB	5
Store (**sw**)	Fetch → Decode → Mem_Adr → Mem_Wr	4
Math (R/I-Type)	Fetch → Decode → Execute → ALU_WB	4
Branch (**beq**)	Fetch → Decode → Branch	3
Jump (**jal**)	Fetch → Decode → Jump	3

4 Component Interfaces

4.1 `processor.v` (Top-Level Wrapper)

Signal	Direction	Width	Description
`clk`	Input	1	System clock
`rst`	Input	1	Active-high synchronous reset

4.2 `control_unit.v`

Signal	Direction	Width	Description
`opcode`	Input	7	Instruction opcode (`instr[6:0]`)
`funct3`	Input	3	Instruction function field (`instr[14:12]`)
`funct7_5`	Input	1	Instruction function field (`instr[30]`)
`zero_flag`	Input	1	Zero flag from ALU for branch evaluation
`pc_write`	Output	1	Enables PC register update
`adr_src`	Output	1	Memory address select (0: PC, 1: `alu_out`)
`mem_write`	Output	1	Enables unified memory write operation
`reg_write`	Output	1	Enables register file write operation
`ir_write`	Output	1	Latch enable for the Instruction Register (`IR`)
`result_src`	Output	2	Selects writeback data (`00`: ALU, `01`: Data, `10`: PC+4)
`alu_src_A`	Output	2	Selects ALU operand A (`00`: PC, `01`: Old PC, `10`: Reg A)
`alu_src_B`	Output	2	Selects ALU operand B (`00`: Reg B, `01`: Imm, `10`: 4)
`imm_src`	Output	3	Selects immediate extension type
`alu_control`	Output	4	Internal ALU operation code

5 Performance & Resource Utilization

The M-RV32I is engineered for applications where silicon area is constrained but multi-cycle execution is acceptable.

5.1 Average CPI Analysis

Using a standard integer workload distribution (e.g., Dhrystone-like blend), we calculate the weighted Cycles Per Instruction (CPI).

Instruction Class	Workload %	Cycles	Contribution
ALU (R-Type / I-Type)	45%	4	1.80
Loads (`lw`)	25%	5	1.25
Stores (`sw`)	10%	4	0.40
Branches (`beq`, etc.)	15%	3	0.45
Jumps (`jal`)	5%	3	0.15
	Average Expected CPI:		4.05

While a CPI of 4.05 is significantly higher than a pipelined processor (CPI $\approx$ 1.0), the multi-cycle design allows for a much higher maximum clock frequency (F_{max}) because the critical path is segmented.

5.2 Estimated Resource Utilization (Xilinx Artix-7)

Synthesis results on a Xilinx Artix-7 (e.g., `xc7a35t`) demonstrate the extreme area efficiency of this shared-resource architecture.

Resource	Usage (Estimate)	Percentage of Target
LUTs (Logic)	$\approx$ 480	$< 2.5\%$
Flip-Flops (Registers)	$\approx$ 250	$< 1.0\%$
BRAM (Unified Memory)	1 Block	Varies by configuration
DSPs	0	0%
Maximum Frequency (F_{max})	$\approx$ 115 MHz	-

Note: Resource usage may vary depending on the synthesis tool's inference of the Register File (LUTRAM vs. Block RAM) and optimization strategies.